Agente Investigador

Valentina Pérez Cadavid

Documentación · Computación Científica Avanzada 2026-II

1. Archivos principales

El proyecto está organizado como una aplicación web (FastAPI) que expone un sistema multiagente apoyado en un modelo servido localmente por Ollama. La siguiente tabla resume el rol de cada archivo del repositorio.

Archivo	Función
main.py	Define la aplicación FastAPI y sus rutas: GET / (healthcheck), POST /generate_report (API en JSON), GET /formulario (muestra el formulario web), POST /generate_report_web (recibe la petición del formulario, llama a planner_agent y luego a investigar_y_escribir por cada sub-tema) y GET /bibliografia (lee referencias_malaria.csv y la muestra en bibliografia.html).
agents.py	Contiene toda la lógica multiagente: el cliente OpenAI apuntando a Ollama (OLLAMA_API_URL), las definiciones de herramientas (wikipedia_tool_def, pubmed_tool_def) y cuatro agentes con roles distintos sobre el mismo modelo: planner_agent (descompone la petición en sub-temas), research_agent (decide y ejecuta tool-calling hacia Wikipedia/PubMed), writer_agent (redacta el resumen a partir de los hallazgos) y editor_agent (evalúa relevancia y fidelidad, devuelve JSON). investigar_y_escribir() orquesta el ciclo escribir → revisar → corregir (máx. 3 intentos).
src/research_tools.py	Implementa las herramientas que el modelo puede invocar: wikipedia_search_tool(query, sentences=5), pubmed_search_tool(query, max_results=3) (consulta la API eutils de NCBI/PubMed) y save_results_to_csv(resultados, source), que agrega cada hallazgo a referencias_malaria.csv.
templates/index.html	Plantilla Jinja2 del formulario principal (/formulario): campo de texto para la petición y bloque donde se muestran los resultados devueltos por generate_report_web.
templates/bibliografia.html	Plantilla Jinja2 que lista las referencias acumuladas en referencias_malaria.csv (ruta /bibliografia).

Archivo	Función
Dockerfile	Imagen basada en python:3.11-slim; copia requirements.txt e instala dependencias, copia el resto del proyecto y expone el puerto 8000; arranca el servidor con uvicorn main:app --host 0.0.0.0 --port 8000.
requirements.txt	Dependencias del proyecto: fastapi, uvicorn, openai (SDK reutilizado para hablar con la API compatible de Ollama), jinja2, python-multipart (requerido por Form()) y requests.
referencias_malaria.csv	Bibliografía persistente generada automáticamente por save_results_to_csv: acumula cada resultado de Wikipedia/PubMed que el agente ha recuperado, con columnas fuente, title, authors, published, journal, url, summary.
test_ollama.py / test_pubmed.py / test_wikipedia.py / test_tool_calling.py / test_research_agents.py / test_save_references.py	Scripts de prueba usados durante el desarrollo para validar por separado cada componente antes de integrarlos: la conexión con Ollama, la herramienta de PubMed, la de Wikipedia, el mecanismo de tool-calling, el ciclo completo de los agentes de investigación y el guardado de referencias en el CSV.

2. Modelo usado

Todos los agentes (planner_agent, research_agent, writer_agent y editor_agent) usan el mismo modelo, qwen3:1.7b, servido localmente mediante Ollama y accedido a través de su API compatible con OpenAI. Se trata de un mismo modelo recibiendo cuatro roles distintos: planificador, investigador, redactor y editor, diferenciados únicamente por el system prompt y la estructura del mensaje en cada función.

La conexión se configura mediante la variable de entorno OLLAMA_API_URL (por defecto http://localhost:11434/v1); al correr la aplicación en Docker, esa variable se apunta a http://host.docker.internal:11434/v1 para que el contenedor pueda alcanzar el servidor de Ollama que corre en el sistema anfitrión (Windows).

qwen3:1.7b es un modelo de lenguaje pequeño (1.7 mil millones de parámetros): corre bien, sin embargo tiene un desempeño notablemente más limitado que modelos grandes en relación al seguimiento de instrucciones, formato estructurado y consistencia de idioma.

3. Fuentes bibliográficas

Algunas referencias acumuladas automáticamente por save_results_to_csv() en referencias_malaria.csv, a partir de las búsquedas que research_agent ejecutó en Wikipedia y PubMed:

Fuente	Título	Autores	Año	Enlace
Wikipedia	Malaria.			ver
PubMed	Prospective Assessment of the Performance of a Combi-Rapid Diagnostic Test to Accurately Diagnose Malaria and Detect C-Reactive Protein in Selected Settings in India. (The American Journal of Tropical Medicine and Hygiene)	Srivastava, B. et al.	2026	ver
PubMed	Use of the "STANDARD G6PD™" quantitative point-of-care test in neonates and infants. (PLoS ONE)	Gornsawun, G. et al.	2026	ver
PubMed	Extrusion-printed silicone microarchitectures for geometry-controlled flow in lateral flow diagnostics and paper microfluidics. (Biosensors & Bioelectronics)	Alioglu, M. A. et al.	2027	ver
PubMed	Aptamer-Based Biosensors for the Detection of Malaria. (Biosensors)	Centelles, J. J.; Imperial, S.	2026	ver

Nota: las fechas de publicación 2026/2027 corresponden a los metadatos devueltos directamente por la API de PubMed (artículos en adelanto de impresión); se reportan tal como las entregó la fuente, sin modificarlas.

4. Limitaciones del modelo

Lo observado al correr el agente deja ver varias limitaciones, tanto del modelo qwen3:1.7b en sí como del diseño del sistema alrededor de él:

- **Fuga de idioma bajo incertidumbre:** en al menos una ejecución, el modelo respondió por completo en chino en lugar de español al no contar con suficiente información para sintetizar el resumen pedido.
- **Fallas de fidelidad en los resúmenes:** el editor_agent rechazó versiones del writer_agent (campo "aprobada": false) por mencionar cifras sin anclarlas claramente a su fuente, o por omitir contenido de estudios con abstracts incompletos, evidencia de que el modelo generador puede perder precisión al sintetizar varias fuentes a la vez.
- **Salida JSON poco confiable:** editor_agent y planner_agent dependen de que el modelo devuelva JSON válido; el propio código incluye un try/except json.JSONDecodeError que trata cualquier respuesta mal formada como "no aprobada" en vez de fallar, lo que confirma que esto ocurre en la práctica.
- **Límite fijo de reintentos:** investigar_y_escribir() permite máximo 3 intentos; si el editor nunca aprueba, se devuelve igualmente la última versión sin aprobar, sin que la interfaz web distinga claramente un resumen aprobado de uno que no lo fue.
- **Información de las fuentes incompleta:** algunos artículos de PubMed llegan sin abstract (summary vacío), lo que limita de entrada lo que el modelo puede sintetizar sobre estos estudios, independientemente de su capacidad.
- **Sin verificación externa de cifras:** el prompt del writer_agent prohíbe inventar o corregir cifras, pero esa instrucción no está respaldada por ningún mecanismo de verificación automática fuera del propio modelo, la fidelidad numérica depende enteramente de que el modelo siga la instrucción.

5. Alcances del modelo

- Elige la herramienta correcta según el tipo de pregunta. research_agent logra distinguir cuándo conviene pubmed_search_tool (literatura científica/papers) de wikipedia_search_tool (definición general), y arma el JSON de argumentos (query, max_results) con el formato que la función espera, trayendo artículos reales y relevantes de PubMed en vez de respuestas genéricas.
- Incorpora la crítica del editor de forma justificada. Cuando editor_agent rechazó una versión, el writer_agent no solo cambió el texto, sino que explicó qué datos excluyó y da su justificación.
- Mantiene las cifras numéricas tal como aparecen en las fuentes, sin modificarlas ni corregirlas por cuenta propia.
- Descompone peticiones amplias en sub-temas concretos y relacionados entre sí con el tema central, en vez de generar categorías genéricas o desconectadas.